ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО	УТВЕРЖДЕНО
Приглашенный преподаватель	Академический руководитель образовательной программы «Программная инженерия», старший преподаватель департамента программной инженерии
_____ С. Б. Гурин	_____ Н. А. Павлочев
«___» _____ 2026 г.	«___» _____ 2026 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

Sentinel — Интеллектуальный тайм-менеджер

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.06.05-01 51 01-1-ЛУ

Исполнители:
Студент группы БПИ236
_____ / Т. А. Петянов /
«___» _____ 2026 г.

Студент группы БПИ236
_____ / Т. С. Максимов /
«___» _____ 2026 г.

Москва 2026

УТВЕРЖДЕН

RU.17701729.06.05-01 51 01-1-ЛУ

Sentinel — Интеллектуальный тайм-менеджер

Программа и методика испытаний

RU.17701729.06.05-01 51 01-1

Листов 51

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

Москва 2026

АННОТАЦИЯ

Настоящий программный документ представляет собой программу и методику испытаний к программному проекту «Sentinel — Интеллектуальный тайм-менеджер».

В разделе «Объект испытаний» указано наименование, краткая характеристика и назначение программы.

В разделе «Цель испытаний» указана цель проведения испытаний.

Раздел «Требования к программе» содержит основные требования к программе, которые подлежат проверке во время испытаний (требования к функционалу и интерфейсу).

Раздел «Требования к программным документам» содержит состав программной документации, которая представляется на испытания.

Раздел «Средства и порядок испытаний» содержит информацию о технических и программных средствах, которые следует использовать во время испытаний, а также порядок этих испытаний.

Раздел «Методы испытаний» содержит информацию об используемых методах испытаний

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 Виды программ и программных документов[1].
2. ГОСТ 19.102-77 Стадии разработки[2].
3. ГОСТ 19.103-77 Обозначения программ и программных документов[3].
4. ГОСТ 19.104-78 Основные надписи[4].
5. ГОСТ 19.105-78 Общие требования к программным документам[5].
6. ГОСТ 19.106-78 Требования к программным документам, выполненным печатным способом[6].
7. ГОСТ 19.301-79 Программа и методика испытаний. Требования к содержанию и оформлению[7].

Изменения к данной Программе и методике испытаний оформляются согласно ГОСТ 19.603-78[8], ГОСТ 19.604-78[9].

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ . . . 5
1.1. Наименование программы . . . 5
1.2. Краткая характеристика области применения программы . . . 5
2. ЦЕЛЬ ИСПЫТАНИЙ . . . 7
3. ТРЕБОВАНИЯ К ПРОГРАММЕ . . . 8
3.1. Требования к функциональным характеристикам . . . 8
3.1.1. Требования к составу выполняемых функций . . . 8
3.1.2. Требования к организации входных данных . . . 15
3.1.3. Требования к организации выходных данных . . . 16
3.1.4. Требования к временным характеристикам . . . 18
3.1.5. Требования к интерфейсу . . . 19
3.1.6. Требования к хранению данных . . . 20
3.1.7. Интерфейсы API . . . 21
3.1.8. Взаимодействие LLM с событиями . . . 23
3.2. Требования к надежности . . . 24
3.2.1. Требования к обеспечению надежного функционирования программы . . . 24
3.2.2. Время восстановления после отказа . . . 25
3.2.3. Отказы из-за некорректных действий оператора . . . 26
3.3. Условия эксплуатации . . . 27
3.3.1. Климатические условия эксплуатации . . . 27
3.3.2. Требования к видам обслуживания . . . 27
3.3.3. Требования к численности и квалификации персонала . . . 27
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ . . . 28
4.1. Состав программной документации . . . 28
4.2. Специальные требования к программной документации . . . 28
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ . . . 29
5.1. Состав технических и программных средств . . . 29
5.2. Порядок проведения испытаний . . . 30
6. МЕТОДЫ ИСПЫТАНИЙ . . . 32
6.1. Проверка требований к программной документации . . . 32
6.2. Автоматизированное тестирование iOS-клиента . . . 33

6.3. Unit-тестирование серверной части . . . 35
6.4. Интеграционное тестирование серверной части . . . 37
6.5. Функциональные проверки серверной части . . . 38
6.5.1. Аутентификация и управление профилем . . . 38
6.5.2. Управление событиями . . . 38
6.5.3. Диалоги, LLM и изображения . . . 39
6.5.4. Ребалансировка расписания . . . 40
6.5.5. Достижения . . . 40
6.5.6. Надёжность и граничные случаи . . . 41
6.6. Тестирование функциональности iOS-клиента . . . 41
6.7. Сквозное функциональное тестирование комплекса . . . 43
6.8. Проверка требований к надёжности . . . 44
6.8.1. Время восстановления после отказа . . . 46
6.8.2. Отказы из-за некорректных действий . . . 46
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ . . . 47
ПРИЛОЖЕНИЕ 2. ТЕРМИНЫ И СОКРАЩЕНИЯ . . . 49
ПРИЛОЖЕНИЕ А. РЕЗУЛЬТАТЫ АВТОМАТИЧЕСКОГО ТЕСТИРОВАНИЯ . . . 51

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Наименование программы: «Sentinel — Интеллектуальный тайм-менеджер».

Наименование программы на английском языке: «Sentinel — AI Scheduler».

Краткое наименование программы для пользователя: «Sentinel».

1.2. Краткая характеристика области применения программы

Программный комплекс Sentinel представляет собой клиент-серверную систему для интеллектуального управления личным расписанием пользователя. Система включает мобильное iOS-приложение, серверную часть с REST API и интеграцию с внешней языковой моделью, а также локальную синхронизацию с календарём устройства через EventKit.

Сквозной сценарий работы системы состоит в следующем: пользователь формулирует запрос на естественном языке в iOS-приложении, при необходимости прикладывает изображения, серверная часть интерпретирует запрос, формирует предложения событий, пользователь подтверждает выбранные действия, после чего изменения сохраняются на сервере и синхронизируются с календарём устройства.

Дополнительная характеристика iOS-клиента.

Разрабатываемое приложение представляет собой мобильную iOS-программу для интеллектуального планирования личного расписания. Основной сценарий работы приложения состоит в преобразовании свободного текстового запроса пользователя, а также прикреплённых изображений, в набор структурированных календарных событий. Полученные предложения пользователь просматривает, при необходимости редактирует через выбор предложений ассистента и затем сохраняет в календарь устройства.

Система учитывает индивидуальные ограничения и контекст пользователя через модель «Resource Battery», реализованную с использованием Foundation Models, поддерживает автоматическую балансировку порядка событий «Rebalance», ведёт историю диалогов и хранит пользовательские настройки. Для взаимодействия с календарём устройства используется EventKit, для хранения локальных записей синхронизации — SwiftData, для хранения пользовательской сессии — Keychain, для базовых настроек — UserDefaults.

Приложение может применяться в следующих сценариях:

1. Планирование учебных занятий, дедлайнов и личных дел на основе свободного текстового описания.

2. Преобразование одного пользовательского запроса в несколько календарных событий с последующим выбором нужных предложений.

3. Обработка фотографий и файлов изображений для извлечения задач, дат и временных ограничений.

4. Анализ занятости пользователя на ближайший день или на выбранный период с помощью «Resource Battery».

5. Ручное управление календарными событиями и синхронизация с календарём устройства.

6. Предварительный просмотр и применение сценария автоматической ребалансировки расписания.

7. Отслеживание пользовательского прогресса через систему достижений.

Дополнительная характеристика серверной части.

Программа «Sentinel — Интеллектуальный тайм-менеджер» функционирует в клиент-серверной архитектуре, где iOS-приложение выступает клиентом, а серверная часть обеспечивает интерпретацию запросов и работу с данными.

Основной сценарий взаимодействия: пользователь в экране «Чат» формулирует запрос на естественном языке. Серверная часть интерпретирует запрос с использованием внешней языковой модели, выполняет поиск по персональным данным (события/напоминания) и возвращает структурированный результат, пригодный для создания или обновления записей календаря на стороне клиента.

Серверная часть интегрируется с внешней языковой моделью для извлечения намерений пользователя и управления событиями, не используя собственные ML-модели. Все данные хранятся за единым REST API с документацией в формате OpenAPI 3.x; история чатов и сообщений поддерживает контекстный диалог. Доступ к персональным данным строго разграничен: каждый пользователь взаимодействует только со своими ресурсами.

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является проверка соответствия программного комплекса Sentinel требованиям технического задания, подтверждение корректности работы iOS-клиента и серверной части как по отдельности, так и в совместном режиме, а также подтверждение того, что программная документация, автоматизированные проверки и фактическая реализация образуют согласованный комплект, пригодный для приемки и дальнейшего сопровождения.

Для iOS-клиента.

Целью испытаний является проверка корректности выполнения программой функций, изложенных в разделе «Требования к программе» документа «Техническое задание», а также подтверждение того, что реализованное приложение, сопровождающая документация и автоматизированные проверки образуют согласованный комплект, пригодный для приемки и дальнейшего сопровождения.

Для серверной части.

Целью испытаний является проверка соответствия серверной части системы «Sentinel — Интеллектуальный тайм-менеджер» требованиям, изложенным в разделе 4 «Требования к программе» документа «Техническое задание» из комплекта документации в соответствии с Единой системой программной документации.

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

3.1. Требования к функциональным характеристикам

3.1.1. Требования к составу выполняемых функций

iOS-приложение должно реализовывать следующие функции:

1. Главный экран «Сегодня»

1.1. Сценарий неавторизованного пользователя

1.1.1. Отображение информационных карточек приложения.

1.1.2. Кнопка входа в аккаунт.

1.1.3. Кнопка создания аккаунта.

1.2. Сценарий авторизованного пользователя

1.2.1. Отображение текущей даты и приветственного заголовка.

1.2.2. Кнопка перехода к экрану профиля.

1.2.3. Карточка расписания на текущий день с количеством событий и краткой информацией о ближайшем событии.

1.2.4. Карточка «Resource Battery» с крупным значением, иконкой состояния и поясняющим текстом.

1.2.5. Индикация состояний «Resource Battery»: hidden, placeholder, setupRequired(.downloadModel), setupRequired(.enableAppleIntelligence), ready.

1.2.6. Кнопка перехода к полному календарю.

1.2.7. Кнопка запуска сценария ребалансировки.

1.2.8. Переход к полному списку достижений.

2. Экран «Чат»

2.1. Работа с диалогом

2.1.1. Отправка текстовых сообщений.

2.1.2. Автоматическое открытие последнего активного чата, если с момента его последнего показа прошло не более 10 минут.

2.1.3. Автоматическое создание нового чата, если правило 10 минут не выполнено.

2.1.4. Загрузка списка чатов и выбор активного диалога.

2.1.5. Загрузка более ранних сообщений.

2.1.6. Отображение ошибок загрузки истории и действие «Повторить».

2.1.7. Сохранение неотправленного сообщения в ленте с индикатором ошибки.

2.1.8. Действия Retry и Remove для сообщения, отправка которого завершилась ошибкой.

2.2. Работа с вложениями

2.2.1. Кнопка добавления вложения.

2.2.2. Открытие модальной панели выбора вложений.

2.2.3. Кнопка All Photos для перехода к полному выбору изображений.

2.2.4. Горизонтальный список последних изображений из фотогалереи.

2.2.5. Кнопка камеры.

2.2.6. Строка Add files для открытия системного файлового пикера.

2.2.7. Однократное согласие на серверную обработку вложений перед первой отправкой изображений или файлов.

2.2.8. Прикрепление фотографий форматов PNG, JPG, JPEG и HEIC из галереи или камеры.

2.2.9. Прикрепление файлов изображений поддерживаемых форматов через системный пикер.

2.2.10. Удаление прикрепленного изображения или файла до отправки сообщения.

2.2.11. Сохранение неотправленного вложения вместе с сообщением для повторной отправки.

2.2.12. Проверка размера и количества вложений до начала отправки.

2.3. Работа с предложениями событий

2.3.1. Отображение блока предложенных событий в сообщении ассистента.

2.3.2. Разворачивание и сворачивание блока предложений.

2.3.3. Выбор одного или нескольких предложений событий.

2.3.4. Кнопка подтверждения выбранных предложений.

2.3.5. Отображение конфликтов с текущим расписанием.

2.3.6. Синхронизация состояния предложений после применения.

3. Экран списка чатов

3.1. Просмотр сохранённых чатов.

3.2. Выбор нужного чата для продолжения диалога.

3.3. Кнопка создания нового чата.

3.4. Кнопка возврата к активному диалогу.

3.5. Удаление чата через жестовое действие или соответствующий UI-контрол.

4. Экран календаря

4.1. Просмотр расписания

4.1.1. Представление недели и месяца.

4.1.2. Открытие и закрытие графического выбора месяца.

4.1.3. Выбор дня в недельной полосе.

4.1.4. Отображение секций по дням.

4.1.5. Отображение day-level индикатора «Resource Battery» для каждой секции дня.

4.2. Управление событиями

4.2.1. Кнопка создания нового события.

4.2.2. Открытие редактора события по нажатию на строку события.

4.2.3. Действия редактирования и удаления события.

4.2.4. Обновление календарных данных после сохранения или удаления события.

5. Экран редактора события

5.1. Поле заголовка события.

5.2. Поле местоположения.

5.3. Поле заметок.

5.4. Переключатель типа сущности: событие или напоминание.

5.5. Переключатель all-day.

5.6. Переключатель fixed event.

5.7. Поля выбора даты и времени начала и окончания.

5.8. Кнопка закрытия редактора.

5.9. Кнопка сохранения события.

6. Экран профиля

6.1. Отображение имени и почты пользователя.

6.2. Debug-only блок Environment, расположенный под профилем и над блоком Default Prompt.

6.3. Поле редактирования Default Prompt.

6.4. Автоматическое сохранение Default Prompt при потере фокуса, закрытии экрана и завершении редактирования.

6.5. Автоматическое добавление Default Prompt к каждому сообщению в чате и к каждому запросу ребалансировки.

6.6. Строка перехода к списку юридических документов.

6.7. Переход на экран достижений.

6.8. Кнопка выхода из учётной записи.

6.9. Кнопка удаления учётной записи.

6.10. Поле ввода текущего пароля при удалении учётной записи.

6.11. Кнопка подтверждения удаления учётной записи.

6.12. Кнопка отмены удаления учётной записи.

7. Экран достижений

7.1. Отображение сводной карточки прогресса.

7.2. Отображение групп достижений и уровней внутри группы.

7.3. Кнопка повторной загрузки данных при ошибке.

7.4. Возможность поделиться разблокированным достижением через системный ShareLink.

8. Экран списка юридических документов

8.1. Отображение списка юридических документов приложения.

8.2. Строки Privacy Policy, Terms of Use, Consent to Personal Data Processing, Attachment Processing Notice и иных обязательных документов.

8.3. Открытие выбранного документа поверх приложения во встроенном Safari controller.

9. Debug-only элемент Environment

9.1. Элемент доступен только в тестовых сборках приложения.

9.2. Элемент расположен на экране Profile под блоком профиля и над блоком Default Prompt.

9.3. Выбор текущего окружения выполняется через встроенное меню со значениями Local, Production и Testing.

10. Экран ребалансировки

10.1. Выбор дней для ребалансировки.

10.2. Отображение количества событий по каждому выбранному дню.

10.3. Отображение day-level индикатора «Resource Battery» для выбранных дней.

10.4. Кнопка предварительного просмотра ребалансировки.

10.5. Экран предварительного просмотра изменений расписания.

10.6. Кнопка обновления предварительного просмотра.

10.7. Кнопка применения изменений.

10.8. Кнопка закрытия экрана ребалансировки.

11. Экран авторизации

11.0. Общие элементы

11.0.1. Кнопка закрытия экрана авторизации.

11.0.2. Отображение ссылок на Privacy Policy и Terms of Use в нижней части экрана.

11.1. Экран входа

11.1.1. Поле ввода email.

11.1.2. Поле ввода пароля.

11.1.3. Кнопка входа.

11.1.4. Кнопка перехода к регистрации.

11.1.5. Кнопка перехода к восстановлению пароля.

11.2. Экран регистрации, шаг 1

11.2.1. Поле ввода email.

11.2.2. Кнопка продолжения.

11.2.3. Кнопка возврата ко входу.

11.2.4. Отображение текста согласия с пользовательскими документами.

11.3. Экран регистрации, шаг 2

11.3.1. Поле ввода пароля.

11.3.2. Поле подтверждения пароля.

11.3.3. Кнопка завершения регистрации.

11.3.4. Кнопка возврата к шагу ввода email.

11.4. Экран подтверждения почты

11.4.1. Поле ввода verification token.

11.4.2. Кнопка подтверждения почты.

11.4.3. Кнопка повторной отправки письма подтверждения.

11.4.4. Кнопка повторной попытки входа после подтверждения.

11.4.5. Кнопка возврата ко входу.

11.5. Экран восстановления пароля

11.5.1. Поле ввода email для отправки письма восстановления.

11.5.2. Кнопка отправки ссылки на восстановление.

11.5.3. Поле ввода reset token.

11.5.4. Поле ввода нового пароля.

11.5.5. Поле подтверждения нового пароля.

11.5.6. Кнопка завершения сброса пароля.

11.5.7. Кнопка возврата ко входу.

Серверная часть должна реализовывать следующие функции.

1. Обработка диалоговых запросов:

1.1. Интеграция с внешней языковой моделью (OpenAI-совместимый API) для извлечения намерений и сущностей из текстовых запросов, включая изображения (vision).

1.2. Оркестрация инструментов (tool-use / function calling): поиск событий, предложение создания/обновления/удаления событий.

1.3. Паттерн propose-before-apply: модель предлагает действия со статусом pending; пользователь подтверждает, после чего изменения применяются.

1.4. Поддержка персональных инструкций пользователя (ai_instructions), влияющих на стиль ответов модели без расширения её роли.

2. Управление календарём:

2.1. Поиск событий по произвольному временному диапазону (date_from / date_to, ISO-8601); без параметров — все предстоящие события.

2.2. Создание, чтение, обновление и удаление событий и напоминаний (напоминание — событие без end_at).

2.3. Идемпотентное создание событий через заголовок X-Idempotency-Key (окно TTL; при повторном запросе возвращается исходный объект без дублирования).

2.4. Батчевая офлайн-синхронизация: применение накопленных создаций/обновлений/удалений одним запросом (upsert-семантика, до 500 операций).

3. Оптимизация расписания (Rebalance):

3.1. Dry-run: LLM получает список событий выбранных дней, учитывает is_fixed и уровень энергии (Resource Battery), предлагает перераспределение без сохранения.

3.2. Apply: пользователь подтверждает подмножество предложений — они атомарно сохраняются.

3.3. Валидация ответа LLM: проверка отсутствия пересечений, корректности временных интервалов, неизменности is_fixed-событий; при провале — один повтор запроса к LLM.

4. Управление диалогами:

4.1. Ведение чат-сессий и сообщений, хранение истории взаимодействий.

4.2. Cursor-based пагинация истории сообщений (параметры before, limit).

4.3. Загрузка изображений в объектное хранилище (S3-совместимое) с последующей передачей URL в LLM vision.

5. Аутентификация и авторизация:

5.1. Регистрация (email + пароль), вход, выдача JWT (HS256, TTL 7 дней).

5.2. Email-верификация аккаунта: токен отправляется письмом, срок действия настраивается (по умолчанию 24 ч).

5.3. Сброс пароля через email-токен (срок действия по умолчанию 1 ч).

5.4. Управление профилем: обновление тайм-зоны, локали, персональных инструкций для LLM.

5.5. Удаление аккаунта с каскадным удалением всех данных пользователя.

6. Совместимость и документация:

6.1. REST API с описанием в OpenAPI 3.x (автогенерация через FastAPI, доступна по /docs).

6.2. Использование ISO-8601 для дат/времени, учёт часового пояса пользователя (IANA) при взаимодействии с LLM.

7. Защита от злоупотреблений:

7.1. Rate limiting на основе user_id (при наличии JWT) или IP-адреса: 30 запросов/мин для чатов и применения ребалансировки, 10 запросов/мин для dry-run ребалансировки.

8. Достижения:

8.1. Выдача пользовательских достижений на основании счётчиков активности (total_events, ai_events, total_reminders, active_days).

8.2. Группы достижений с уровнями; API возвращает все группы с текущим прогрессом и статусом разблокировки каждого уровня.

Требования к взаимодействию компонентов.

1. Обмен данными между iOS-клиентом и серверной частью должен выполняться по REST API с префиксом `/api/v1` в формате JSON по HTTPS, а для загрузки изображений — в формате multipart/form-data.
2. Аутентификация между клиентом и сервером должна выполняться через JWT в заголовке `Authorization: Bearer <token>`.
3. Клиент должен передавать на сервер сообщения пользователя, данные профиля, параметры календарных сущностей, запросы на apply предложений и запросы на ребалансировку.
4. Сервер должен возвращать клиенту структурированный результат в виде JSON, пригодный для визуализации предложений событий, обновления истории чата и синхронизации состояния календаря.
5. После применения предложений или ребалансировки клиент должен повторно синхронизировать календарные данные и отдельно поддерживать экспорт в EventKit без дублирования событий.

6. Серверное состояние событий и локальная синхронизация EventKit рассматриваются как отдельные контуры; наличие локального события не должно подменять серверный источник истины.
7. Часовые пояса пользователя должны передаваться и интерпретироваться согласованно: клиент отправляет даты и время в ISO-8601, сервер хранит временные значения в UTC и учитывает IANA-тайм-зону пользователя при логике LLM и профиля.

3.1.2. Требования к организации входных данных

Входные данные iOS-клиента.

Входные данные представляют собой:

1. Сообщения в чате
   1.1. Текст до 2000 символов в кодировке UTF-8.
   1.2. Фотографии форматов PNG, JPG, JPEG и HEIC; до 10 МБ на файл; до 10 изображений на сообщение.
   1.3. Файлы изображений поддерживаемых форматов через системный пикер.
2. Данные авторизации и управления учётной записью
   2.1. Адрес электронной почты.
   2.2. Пароль пользователя.
   2.3. Verification token.
   2.4. Reset token.
3. Данные календарных сущностей
   3.1. Заголовок события.
   3.2. Местоположение.
   3.3. Описание.
   3.4. Дата и время начала.
   3.5. Дата и время окончания.
   3.6. Признаки all-day, fixed, reminder или event.
4. Данные ребалансировки и пользовательского контекста
   4.1. Выбранные для ребалансировки дни.
   4.2. Значение Default Prompt.
   4.3. Текущая временная зона устройства.
   4.4. Выбранное окружение приложения в тестовой сборке.
5. Правила приёма и валидации
   5.1. Проверка длины сообщения и допустимых символов.

5.2. Проверка типа, размера и количества вложений.

5.3. Проверка непустого заголовка события перед сохранением.

5.4. Проверка обязательных полей сценариев регистрации, подтверждения почты и восстановления пароля.

5.5. Запрос однократного согласия на серверную обработку вложений перед первой отправкой файлов или изображений.

Входные данные серверной части.

1. Формат обмена: JSON по HTTPS; для загрузки файлов — multipart/form-data.

2. Аутентификация: заголовок Authorization: Bearer (для защищённых эндпоинтов).

3. Идемпотентность: необязательный заголовок X-Idempotency-Key (строка до 255 символов) при создании события.

4. Даты/время: ISO-8601 с обязательным учётом пользовательской тайм-зоны (IANA) на уровне сервиса; хранение в UTC.

5. События/напоминания: title (строка, 1–255 символов), start_at (ISO-8601), end_at (ISO-8601 или NULL для напоминаний; должно быть строго позже start_at), all_day (bool), type (event/reminder), location (строка, опц.), is_fixed (bool), source (user/ai/import).

6. Загрузка изображений: PNG, JPG, JPEG, HEIC, максимальный размер 10 МБ.

7. Ограничения: валидация схемы входных данных (Pydantic v2); проверка прав доступа к ресурсам пользователя; батчевая синхронизация — не более 500 записей в каждом из списков upserts и deletes.

Входные данные взаимодействия компонентов.

1. Пользовательские сообщения, вложения и действия подтверждения должны передаваться с клиента на сервер без потери контекста активного чата и выбранных предложений.
2. Данные календарных сущностей и временные интервалы должны передаваться в формате, совместимом с серверной валидацией и последующей локальной синхронизацией через EventKit.
3. Значение Default Prompt, временная зона и иные пользовательские настройки, влияющие на серверную интерпретацию запроса, должны передаваться согласованно и не должны теряться при повторных запросах, перезапуске приложения и смене активного чата.

3.1.3. Требования к организации выходных данных

Выходные данные iOS-клиента.

1. Внутренние сущности

1.1. Пользователь — владелец учётной записи и персональных настроек.

1.2. Чат — отдельный диалог пользователя с ассистентом.

1.3. Сообщение чата — отдельная запись диалога с текстом, изображениями и, при наличии, структурированными предложениями событий.

1.4. Событие — запись расписания с идентификатором, временем, параметрами и происхождением.

1.5. Достижение — запись о прогрессе пользователя, уровне и статусе разблокировки.

2. Внешние артефакты

2.1. Ответы серверной части в формате JSON по контрактам REST API.

2.2. События календаря устройства, синхронизированные через EventKit.

2.3. Юридические документы, отображаемые внутри приложения.

2.4. Служебные сообщения о статусе операций и ошибках.

3. Локально сохраняемые данные

3.1. Записи синхронизации календаря в SwiftData.

3.2. Пользовательская сессия в Keychain.

3.3. AppSettings и service-флаги в UserDefaults.

4. Требования к целостности выходных данных

4.1. Исключение дублирования событий при экспорте в календарь устройства.

4.2. Сохранение признака конфликта по времени для предложенных и существующих событий.

4.3. Сохранение связи между серверными идентификаторами событий и локальными событиями календаря.

Выходные данные серверной части.

1. Формат ответов: JSON; коды ответов HTTP соответствуют семантике (2xx/4xx/5xx).

2. Ответы поиска событий: объект {items: […], total: int}; каждый элемент содержит id, user_id, title, description, start_at, end_at, all_day, type, location, is_fixed, source, created_at, updated_at.

3. Ответы по чатам и сообщениям: список сообщений в формате {items: […], has_more: bool} (cursor-based пагинация); каждое сообщение содержит id, chat_id, role, content_text, content_structured, ai_model, created_at.

4. Структурированный контент сообщений (content_structured): JSON-объект с дискриминатором type; тип event_actions содержит массив предложенных действий (create/update/delete) со статусами pending/accepted/rejected.

5. Ответ ребалансировки: {proposed: […], summary: строка, changed_count: int, unchanged_count: int}; каждый элемент proposed содержит id, title, start_at, end_at, original_start_at, original_end_at, changed.

6. Достижения: {groups: […]}; каждая группа содержит group_code, category, counter_name, current_value, levels с полями level, title, description, icon, target_value, unlocked, earned_at.

7. Идентификаторы: UUID v4 (строка).

8. Ошибки: единый формат {code, message, details}; code — строковый код (BAD_REQUEST, UNAUTHORIZED, FORBIDDEN, NOT_FOUND, CONFLICT, VALIDATION_ERROR, RATE_LIMIT_EXCEEDED, INTERNAL_ERROR).

Требования к целостности и синхронизации выходных данных компонентов.

1. Исключение дублирования событий при экспорте в календарь устройства.
2. Сохранение связи между серверными идентификаторами событий и локальными идентификаторами EventKit.
3. Сохранение признака временного конфликта для предложенных и существующих событий.
4. Согласованность статусов предложений событий между серверной частью и отображением клиента после apply-операций.
5. Изоляция пользовательских данных: ни один пользователь не должен получать чужие события, чаты, достижения или токены.

3.1.4. Требования к временным характеристикам

Требования к временным характеристикам iOS-клиента.

1. Реакции интерфейса, не требующие сетевого запроса, должны выполняться без визуально заметной задержки для пользователя.
2. Переключение экранов и отображение листов не должно сопровождаться зависанием интерфейса.
3. Целевое время интерпретации пользовательского запроса на серверной стороне составляет 3–6 секунд; предельное время ожидания ответа — 15 секунд.
4. Операция создания или обновления календарного события через UI приложения должна выполняться в пределах нормального пользовательского ожидания без блокировки интерфейса.

5. Автосохранение Default Prompt должно происходить при завершении редактирования без дополнительной кнопки сохранения.

Требования к временным характеристикам серверной части.

1. Время ответа API для простых операций (чтение списков/деталей): ≤ 500 мс при номинальной нагрузке.

2. Поиск событий в БД (диапазон + текст): медиана ≤ 200–300 мс.

3. Синхронный диалоговый запрос с LLM: конечный ответ ≤ 8 с; при превышении — асинхронный сценарий/уточнение.

4. Вставка/обновление события в БД: ≤ 300 мс (без внешних вызовов).

3.1.5. Требования к интерфейсу

Требования к интерфейсу iOS-клиента.

1. Общие требования

1.1. Соответствие Apple Human Interface Guidelines.

1.2. Поддержка светлой и тёмной темы оформления.

1.3. Поддержка Dynamic Type.

1.4. Минимальный размер интерактивных элементов — 44×44 pt.

1.5. Поддержка VoiceOver и корректных accessibility-меток.

1.6. Контраст визуальных элементов не ниже требований WCAG 2.1 AA.

2. Организация экранов

2.1. Основной пользовательский сценарий «чат → предложения событий → подтверждение → календарь» должен быть доступен без перехода в служебные разделы.

2.2. Пользовательские настройки должны быть сосредоточены на экране Profile.

2.3. Юридические документы должны открываться модально поверх приложения.

2.4. Environment должен быть доступен только в тестовых сборках.

3. Индикация состояний

3.1. Отображение статуса загрузки для сетевых операций.

3.2. Отображение текстовых ошибок и действий повторной попытки.

3.3. Отображение состояний «Resource Battery».

3.4. Отображение ошибок неуспешной отправки сообщений с возможностью Retry или Remove.

Требования к интерфейсу серверной части.

Версионирование REST: префикс /api/v1.

3.1.6. Требования к хранению данных

Хранение данных на стороне iOS-клиента.

1. Записи синхронизации календаря в SwiftData.
2. Пользовательская сессия в Keychain.
3. AppSettings и service-флаги в UserDefaults.

Хранение данных на стороне серверной части.

Система должна обеспечивать надёжное хранение следующих сущностей:

1. Пользователи (таблица users): уникальный UUID-идентификатор; email (уникальный, индекс); хеш пароля (bcrypt); флаг подтверждения email (is_verified); тайм-зона (IANA, по умолчанию Europe/Moscow); локаль (по умолчанию ru-RU); предпочтения (JSONB); персональные инструкции для LLM (ai_instructions, TEXT, nullable, до 500 символов); временные метки создания, обновления и последнего входа.

2. События и напоминания (таблица events): UUID; связь с пользователем (CASCADE DELETE); заголовок (до 255 символов); описание (TEXT); start_at (TIMESTAMPTZ, индекс); end_at (TIMESTAMPTZ, NULL для напоминаний); признак события на весь день (all_day); тип (event/reminder); местоположение (до 255 символов); признак фиксированного времени (is_fixed); источник создания (user/ai/import); временные метки создания и обновления.

3. Чаты (таблица chats): UUID; связь с пользователем (CASCADE DELETE); заголовок (до 255 символов); временная метка последнего сообщения (last_message_at); временные метки создания и обновления.

4. Сообщения чатов (таблица chat_messages): UUID; связь с чатом (CASCADE DELETE); роль отправителя (user/assistant/tool/system); текстовое содержимое (TEXT); структурированное содержимое (JSONB; тип event_actions — список предложений LLM, тип image_message — вложения изображений); идентификатор модели ИИ; временная метка создания.

5. Достижения (таблица achievements): UUID; группа (group_code); уровень (level, 1–3); заголовок; описание; иконка; категория; название счётчика (counter_name); целевое значение (target_value). Уникальное ограничение на пару (group_code, level).

6. Выданные достижения (таблица user_achievements): UUID; связь с пользователем (CASCADE DELETE); связь с достижением; временная метка выдачи. Уникальное ограничение на пару (user_id, achievement_id).

7. Счётчики пользователя (таблица user_counters): UUID; связь с пользователем (CASCADE DELETE); название счётчика (total_events, ai_events, total_reminders, active_days); значение (integer ≥ 0). Уникальное ограничение на пару (user_id, counter_name).

8. Ключи идемпотентности (таблица idempotency_keys): UUID; ключ (до 255 символов); связь с пользователем (CASCADE DELETE); связь с событием (CASCADE DELETE); срок действия (expires_at); временная метка создания. Уникальное ограничение на пару (key, user_id).

9. Токены аутентификации (таблица auth_tokens): UUID; связь с пользователем (CASCADE DELETE); токен (уникальный, индекс); тип токена (email_verification / password_reset); срок действия; временная метка создания.

10. Требования к организации хранения:

10.1. UUID v4 как первичный ключ для всех сущностей.

10.2. TIMESTAMPTZ (с тайм-зоной) для всех временных меток; хранение в UTC.

10.3. Каскадное удаление: при удалении пользователя удаляются все его данные.

10.4. Поиск событий по временным диапазонам с медианным временем отклика не более 300 мс (индекс на start_at).

10.5. Поддержка хранения структурированных данных через JSONB (content_structured, preferences).

10.6. Обеспечение уникальности критичных полей: email пользователя; пара (group_code, level) в достижениях; пара (user_id, achievement_id) в выданных достижениях; пара (key, user_id) в ключах идемпотентности.

3.1.7. Интерфейсы API

1. Аутентификация и управление аккаунтом:

1.1. POST /api/v1/auth/register — регистрация (email, пароль, тайм-зона); возвращает объект пользователя; при включённой верификации отправляет письмо.

1.2. POST /api/v1/auth/login — вход; возвращает JWT-токен.

1.3. POST /api/v1/auth/verify-email — подтверждение email по токену из письма.

1.4. POST /api/v1/auth/resend-verification — повторная отправка письма подтверждения.

1.5. POST /api/v1/auth/forgot-password — запрос письма со ссылкой сброса пароля.

1.6. POST /api/v1/auth/reset-password — установка нового пароля по токену из письма.

1.7. GET /api/v1/auth/me — профиль текущего пользователя (требует JWT).

1.8. PATCH /api/v1/auth/me — обновление профиля: timezone, locale, ai_instructions (требует JWT).

1.9. DELETE /api/v1/auth/me — удаление аккаунта с подтверждением пароля; каскадно удаляет все данные пользователя (требует JWT).

2. Диалоги/чаты:

2.1. GET /api/v1/chats — список чатов текущего пользователя, отсортированных по last_message_at.

2.2. POST /api/v1/chats — создать чат-сессию (title).

2.3. DELETE /api/v1/chats/{id} — удалить чат вместе со всеми сообщениями; ответ 204.

2.4. GET /api/v1/chats/{id}/messages — история сообщений с cursor-based пагинацией (параметры limit, before); ответ {items, has_more}.

2.5. POST /api/v1/chats/{id}/messages — отправить сообщение; если role=«user» — вызывает LLM и возвращает ответ ассистента.

2.6. POST /api/v1/chats/{id}/messages/{msg_id}/apply — применить действия из сообщения ассистента: принять (accepted_indices) или отклонить предложения; выполняет соответствующие операции с событиями.

2.7. POST /api/v1/chats/{id}/upload — загрузить изображение (multipart/form-data); возвращает {url, filename, mime_type}.

3. События и напоминания:

3.1. GET /api/v1/events — поиск событий; параметры date_from, date_to (ISO-8601, оба опциональны); без параметров — все предстоящие события.

3.2. POST /api/v1/events — создать событие/напоминание; поддерживает заголовок X-Idempotency-Key.

3.3. GET /api/v1/events/{id} — получить событие по идентификатору.

3.4. PATCH /api/v1/events/{id} — обновить событие/напоминание (частичное обновление).

3.5. DELETE /api/v1/events/{id} — удалить событие/напоминание; ответ 204.

3.6. POST /api/v1/events/sync — батчевая офлайн-синхронизация: upsert-список и список id на удаление (до 500 элементов каждый); возвращает {created, updated, deleted}.

4. Оптимизация расписания:

4.1. POST /api/v1/events/rebalance — dry-run: LLM предлагает перераспределение событий для выбранных дней с учётом is_fixed и resource_battery; ничего не сохраняет; ответ {proposed, summary, changed_count, unchanged_count}.

4.2. POST /api/v1/events/rebalance/apply — применить подмножество предложений из dry-run; ответ 204.

5. Достижения:

5.1. GET /api/v1/achievements — все группы достижений с текущим прогрессом и статусом разблокировки каждого уровня.

6. Системные:

6.1. GET /api/v1/health — проверка работоспособности сервиса; ответ {status: «healthy»}.

Примечания:

1. Версионирование API — /api/v1; контракты описываются в OpenAPI 3.x (Swagger UI доступен по /docs). Продуктивная среда планируется к развёртыванию по адресу `https://sentinel-ai.tech`; документация API будет доступна по `https://sentinel-ai.tech/docs`.
2. Все защищённые эндпоинты принимают JWT в заголовке Authorization: Bearer .
3. Даты/время — ISO-8601; хранение в UTC; конвертация в тайм-зону пользователя выполняется на уровне LLM-сервиса.
4. Единый формат ошибок: {code, message, details}.

3.1.8. Взаимодействие LLM с событиями

1. Механизм: LLM взаимодействует с данными пользователя через tool-use (function calling) — бэкенд предоставляет модели четыре инструмента: search_events, create_event, update_event, delete_event.

2. Паттерн propose-before-apply: инструменты create_event, update_event, delete_event не изменяют данные немедленно — каждый вызов формирует объект EventAction со статусом pending. Пользователь видит список предложений и подтверждает через POST /api/v1/chats/{id}/messages/{msg_id}/apply; только тогда изменения сохраняются.

3. Инструмент create_event — обязательные поля аргументов:

3.1. title: строка (не пустая).

3.2. start_at: строка ISO-8601 с обязательным смещением тайм-зоны пользователя.

4. Инструмент create_event — дополнительные поля аргументов:

4.1. end_at: строка ISO-8601; при наличии должно быть строго позже start_at.

4.2. all_day: boolean (по умолчанию false).

4.3. type: enum {event, reminder} (по умолчанию event).

4.4. description: строка (опц.).

4.5. location: строка (опц.).

4.6. is_fixed: boolean (по умолчанию false).

5. Поле source у событий, созданных через LLM, устанавливается в значение «ai».

6. Инструмент update_event: обязателен event_id (UUID реального события, полученного из search_events). Модель не должна угадывать или создавать UUID.

7. Инструмент delete_event: обязателен event_id. Перед вызовом модель обязана выполнить search_events.

8. Валидация: ISO-8601 для дат/времени; end_at > start_at; нормализация наивного datetime к тайм-зоне пользователя (IANA) на уровне сервиса.

9. Поведение модели: при запросе на создание — немедленно предлагает событие с разумными допущениями (например, длительность 1 ч); уточняющие вопросы задаёт после предложения. Временные указания пользователя интерпретируются в его IANA-тайм-зоне из профиля — без вывода тайм-зоны из названий мест. Запросы вне темы управления календарём отклоняются.

3.2. Требования к надежности

3.2.1. Требования к обеспечению надежного функционирования программы

Требования к iOS-клиенту.

1. Устойчивость работы приложения

1.1. Валидация входных данных перед запуском критических операций.

1.2. Контролируемая обработка сетевых ошибок, ошибок API и локальных отказов без нарушения целостности пользовательского состояния.

1.3. Идемпотентность операций синхронизации календаря для предотвращения дублирования событий.

1.4. Сохранение секретов и токенов в защищённом хранилище Keychain.

1.5. Сохранение локальных настроек пользователя вне volatile-состояния UI.

1.6. Сохранение видимости конфликтов расписания для пользователя.

Требования к серверной части.

1. Идемпотентность операций создания событий: поддержка заголовка X-Idempotency-Key с хранением ключей в БД (PostgreSQL UNIQUE + TTL). При повторном запросе с тем же ключом возвращается исходный объект события без создания дубликата.

2. Retry при провале LLM в ребалансировке: при невалидном ответе модели выполняется один повтор; при повторном провале — 422.

3. Graceful degradation: при недоступности LLM возвращается 503 с понятным сообщением. При недоступности хранилища изображений — 503.

4. Контроль целостности данных: транзакции БД для всех мутирующих операций; валидация входных данных через Pydantic v2.

5. Rate limiting: 30 запросов/мин для POST /chats/{id}/messages и POST /events/rebalance/apply; 10 запросов/мин для POST /events/rebalance. Ключ лимита — user_id из JWT (при наличии валидного токена) или IP.

6. Логирование: структурированные логи в stdout; каждый HTTP-запрос логируется с методом, путём, статусом и временем выполнения; ключевые события (создание/изменение/удаление событий, синхронизация, ребалансировка) логируются отдельно.

7. Целевое значение доступности сервиса — не ниже 99.5% (для учебного проекта — как ориентир).

Требования к взаимодействию компонентов.

1. Пользовательские действия не должны приводить к рассогласованию серверного состояния и локального календаря без возможности повторной синхронизации.

2. Источник истины для серверных событий — backend; локальная синхронизация EventKit должна только отражать согласованное серверное состояние.

3. Ошибки авторизации, валидации и доступа должны корректно передаваться клиенту и отображаться пользователю без аварийного завершения приложения.

3.2.2. Время восстановления после отказа

Требования к iOS-клиенту.

1. Восстановление пользовательского контекста

1.1. Автоматическое восстановление пользовательской сессии при наличии сохранённого токена.

1.2. Восстановление Default Prompt и выбранного окружения приложения.

1.3. Восстановление идентификатора недавно активного чата в пределах окна актуальности.

1.4. Повторная загрузка календарных событий и серверных данных после перезапуска приложения.

Требования к серверной части.

При отказе отдельного компонента (базы данных, LLM-провайдера, хранилища изображений) восстановление в штатный режим работы выполняется перезапуском контейнеров командой `docker compose restart`. Целевое время восстановления — не более 5 минут.

При полном отказе сервера восстановление производится из резервной копии базы данных. Целевое время восстановления — не более 30 минут при наличии актуальной резервной копии.

3.2.3. Отказы из-за некорректных действий оператора

Требования к iOS-клиенту.

1. Некорректные действия пользователя в auth-сценариях

1.1. Проверка обязательности email и пароля при входе.

1.2. Проверка обязательности verification token при подтверждении почты.

1.3. Проверка обязательности reset token и новых учётных данных при восстановлении пароля.

2. Некорректные действия пользователя при работе с сообщениями и вложениями

2.1. Проверка ограничения размера одного вложения.

2.2. Проверка ограничения общего количества вложений в сообщении.

2.3. Проверка сохранения ошибочно неотправленного сообщения в управляемом состоянии с возможностью Retry или Remove.

3. Некорректные действия пользователя при работе с календарём и профилем

3.1. Проверка обязательности непустого заголовка события перед сохранением.

3.2. Проверка обязательности текущего пароля при удалении учётной записи.

3.3. Проверка корректного поведения при закрытии экрана с редактируемым Default Prompt.

Требования к серверной части.

Программа должна обеспечивать защиту от некорректных действий пользователей и операторов:

1. Некорректные HTTP-запросы (ошибки формата, отсутствие обязательных полей, неверный тип данных) возвращают код 422 с подробным описанием ошибки в формате {code, message, details}.

2. Несанкционированные запросы (отсутствие токена или истёкший JWT) возвращают код 401.

3. Обращение к чужим ресурсам (события, чаты другого пользователя) возвращают код 403.

4. Обращение к несуществующим ресурсам возвращает код 404.

5. Превышение лимита запросов возвращает код 429.

6. Никакое некорректное действие пользователя не приводит к потере данных других пользователей или нарушению целостности базы данных.

3.3. Условия эксплуатации

3.3.1. Климатические условия эксплуатации

Для iOS-клиента применимы штатные условия эксплуатации мобильных устройств Apple.

Применимы стандартные условия эксплуатации серверного программного обеспечения; климатические требования определяются условиями размещения серверной инфраструктуры и провайдера облака.

3.3.2. Требования к видам обслуживания

Требования к iOS-клиенту.

1. Регулярное обновление зависимостей iOS-проекта и используемых средств сборки, тестирования и документации.

Требования к серверной части.

1. Регулярное обновление зависимостей и установленных пакетов.

2. Мониторинг логов и основных метрик производительности.

3. Резервное копирование БД согласно регламенту (ежедневно, с возможностью восстановления до точки во времени).

3.3.3. Требования к численности и квалификации персонала

Требования к iOS-клиенту.

1. iOS-разработчик с опытом работы со Swift, SwiftUI, TCA, EventKit и средствами тестирования Xcode.

Требования к серверной части.

1. Backend-разработчик с опытом разработки веб-сервисов и работы с реляционными СУБД.

2. При необходимости — DevOps-специалист для CI/CD и развёртывания.

3. Пользователь конечного приложения специальных навыков администрирования не требует.

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

4.1. Состав программной документации

1. «Sentinel — Интеллектуальный тайм-менеджер». Техническое задание (ГОСТ 19.201-78).
2. «Sentinel — Интеллектуальный тайм-менеджер». Программа и методика испытаний (ГОСТ 19.301-79).
3. «Sentinel — Интеллектуальный тайм-менеджер». Пояснительная записка (ГОСТ 19.404-79).
4. «Sentinel — Интеллектуальный тайм-менеджер». Руководство оператора (ГОСТ 19.505-79).
5. «Sentinel — Интеллектуальный тайм-менеджер». Текст программы (ГОСТ 19.401-78).

4.2. Специальные требования к программной документации

Документы должны быть выполнены в соответствии с ГОСТ 19.106-78 [6] и ГОСТами к каждому виду документа (см. п. 5.1).

Пояснительная записка должна быть загружена в систему Антиплагиат через LMS «НИУ ВШЭ».

Техническое задание и пояснительная записка, титульные листы других документов должны быть подписаны руководителем разработки и исполнителем.

Документация и программа сдаётся в электронном виде в формате .pdf или .docx в архиве .zip или .rar.

За две недели до защиты все материалы курсового проекта должны быть загружены в проект дисциплины «Курсовой проект» в SmartLMS НИУ ВШЭ.

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Состав технических и программных средств

Технические и программные средства для испытаний iOS-клиента.

Для проведения испытаний использовались следующие технические и программные средства:

1. Технические средства

1.1. Компьютер Apple Mac с установленной средой разработки Xcode.

1.2. Симулятор iPhone 17 Pro с версией iOS 26.2.

1.3. Файловая система разработки с локальной копией репозитория проекта Sentinel.

2. Программные средства

2.1. Xcode 26.2.

2.2. Схема проекта «Sentinel».

2.3. Средства запуска и анализа автоматизированных тестов Xcode.

2.4. Фреймворк Swift Testing.

2.5. TCA TestStore для проверки переходов состояний.

2.6. Средство анализа покрытия кода xccov.

3. Испытуемые артефакты

3.1. Исходный код приложения Sentinel, включая основной таргет приложения и локальный Swift package SentinelModules с библиотекой SentinelUI.

3.2. Документы «Техническое задание», «Пояснительная записка», «Руководство оператора» и «Программа и методика испытаний».

3.3. Автоматизированный тестовый набор таргета SentinelTests.

Технические и программные средства для испытаний серверной части.

Технические требования к окружению:

1. CPU: не менее 2 vCPU.

2. RAM: не менее 4 ГБ.

3. Дисковое пространство: не менее 20 ГБ.

4. Доступ в сеть Интернет (для LLM API).

Программные требования:

1. ОС: Linux Ubuntu 20.04 LTS и выше или macOS 13+.

2. Docker 24.0+ и Docker Compose 2.20+.

3. Python 3.11+ с виртуальным окружением venv.

4. pytest 8.0+, pytest-asyncio, httpx.

5. Переменные окружения из файла .env (DATABASE_URL, SECRET_KEY, LLM_API_KEY и др.).

Технические и программные средства для сквозных испытаний комплекса.

1. Локальные копии репозиториев iOS-приложения, backend-проекта и комплекта документации.
2. Доступ к REST API, OpenAPI / Swagger UI и к артефактам автоматизированного тестирования обеих частей системы.
3. Рабочее окружение, позволяющее параллельно запускать iOS-симулятор, серверные контейнеры и средства анализа результатов испытаний.

5.2. Порядок проведения испытаний

Порядок проведения испытаний iOS-клиента.

Испытания проводились в следующем порядке:

1. Анализ действующего комплекта проектной документации и проверка согласованности терминологии.
2. Сопоставление требований документа «Техническое задание» с реальными представлениями, редьюсерами, клиентами и тестами проекта.
3. Запуск автоматизированного тестового набора проекта на симуляторе iPhone 17 Pro с iOS 26.2 через Xcode 26.2.
4. Анализ состава пройденных тестов и итогового покрытия кода.
5. Сопоставление результатов автоматизированных проверок с основными пользовательскими сценариями Sentinel.
6. Фиксация результатов испытаний, выявленных ограничений и областей дальнейшего наращивания тестовой базы.

Порядок проведения испытаний серверной части.

Испытания проводятся в следующем порядке:

1. Проверка требований к программной документации.

2. Автоматическое unit-тестирование (`pytest tests/unit/`).

3. Автоматическое интеграционное тестирование (`pytest tests/integration/` при запущенном Docker Compose).

4. Функциональные проверки через Swagger UI по адресу /docs.

5. Проверка требований к надёжности.

Порядок проведения сквозных испытаний комплекса.

1. Согласование комплекта требований, терминологии и API-контрактов между клиентской и серверной частями.
2. Подготовка общего испытательного окружения, в котором доступны iOS-клиент, backend-сервис, база данных и документация API.
3. Последовательное выполнение клиентских, серверных и сквозных сценариев с фиксацией результатов и отклонений.
4. Сопоставление результатов автоматизированных и ручных проверок с основным сценарием «чат → предложения событий → подтверждение → календарь».

6. МЕТОДЫ ИСПЫТАНИЙ

6.1. Проверка требований к программной документации

Проверка требований к программной документации iOS-клиента.

Проверка требований к программной документации выполнялась путём анализа состава документов, согласованности их структуры и соответствия их содержания текущему продукту Sentinel.

1. Проверка состава документации

1.1. Проверено наличие технического задания.

1.2. Проверено наличие пояснительной записки.

1.3. Проверено наличие руководства оператора.

1.4. Проверено наличие текста программы.

1.5. Проверено наличие программы и методики испытаний.

2. Проверка согласованности наименований и терминологии

2.1. Проверено единое использование наименования «Sentinel — Интеллектуальный тайм-менеджер».

2.2. Проверено единое использование английского наименования «Sentinel — AI Scheduler».

2.3. Проверено единое использование краткого наименования «Sentinel».

2.4. Проверено единое понимание терминов «Profile», «Default Prompt», «Resource Battery», «Rebalance», «Environment».

3. Проверка содержательной согласованности

3.1. Проверено, что пользовательские настройки сосредоточены на экране Profile.

3.2. Проверено, что legal-flow описан через список документов и модальное открытие документа внутри приложения.

3.3. Проверено, что Environment описан как debug-only функция тестовой сборки.

3.4. Проверено, что Default Prompt описан как автоматически сохраняемый и автоматически добавляемый к сообщениям и запросам ребалансировки.

4. Результаты проверки

4.1. Комплект основных документов по проекту присутствует.

4.2. Наименования программы и основные продуктовые термины приведены к согласованному виду.

4.3. Настоящий документ синхронизирован с техническим заданием, пояснительной запиской, руководством оператора и фактическим кодом приложения.

Проверка требований к программной документации серверной части.

Проверка осуществляется визуальным контролем наличия и оформления документов:

1. Наличие всех пяти документов комплекта (ТЗ, ПЗ, ПМИ, ТП, РО).

2. Соответствие структуры каждого документа требованиям соответствующего ГОСТ.

3. Наличие листов утверждения с подписями руководителя и исполнителя.

4. Наличие списка литературы и приложений во всех документах.

5. Загрузка пояснительной записки в систему Антиплагиат через LMS НИУ ВШЭ.

Проверка требований к программной документации программного комплекса.

1. Проверялась согласованность терминологии, наименования программы и описания сквозных сценариев между документами по iOS-клиенту, backend-компоненту и общему PMI.
2. Проверялась согласованность состава функций, API-контрактов, правил синхронизации и требований к пользовательским сценариям.
3. Проверялось отсутствие противоречий между локальным контуром EventKit и серверным источником истины по событиям.

6.2. Автоматизированное тестирование iOS-клиента

Автоматизированные испытания проводились штатными средствами Xcode по схеме «Sentinel» на симуляторе iPhone 17 Pro с iOS 26.2. Для написания проверок используется Swift Testing. Для тестирования редьюсеров The Composable Architecture используется TestStore, позволяющий проверять начальное состояние, последовательность действий, асинхронные эффекты и итоговое состояние модуля.

1. Тесты сетевых контрактов и транспортного слоя

1.1. Файл «APIRequestAndErrorTests.swift» проверяет построение HTTP-запросов, формирование заголовков, query-параметров и декодирование ошибок API.

1.2. Файл «DTOCodingTests.swift» проверяет кодирование и декодирование DTO авторизации, событий, чатов, сообщений, достижений и структурированного контента.

1.3. Файл «APIModelConverterTests.swift» проверяет преобразование транспортных DTO в доменные сущности приложения, включая обработку fallback-ветвей.

2. Тесты сессии и настроек приложения

2.1. Файл «SessionModelTests.swift» проверяет корректность сериализации и десериализации аутентифицированной пользовательской сессии.

2.2. Файл «AppConfigurationAndSettingsTests.swift» проверяет адреса окружений, юридические

документы, поведение AppSettings, сохранение и повторную загрузку пользовательских настроек.

2.3. Файл «DefaultPromptEnvelopeTests.swift» проверяет добавление и удаление обёртки Default Prompt в содержимом сообщений.

3. Тесты доменных моделей и производных состояний

3.1. Файл «HomeStateDerivedTests.swift» проверяет вычисляемые свойства домашнего экрана, карточки расписания и «Resource Battery».

3.2. Файл «CalendarStateDerivedTests.swift» проверяет построение payload редактора события, формирование секций повестки дня, вычисление конфликтов и day-level battery request.

3.3. Файл «AchievementAndAuthStateTests.swift» проверяет производные свойства auth-сценария и модели достижений.

3.4. Файл «ChatDomainModelTests.swift» проверяет доменные модели чата, предложений событий, сообщений и вложений.

3.5. Файл «ChatThreadSupportTests.swift» проверяет вспомогательную логику формирования заголовков чатов, объединения сообщений и обновления предложений.

3.6. Файлы «AdditionalPureCoverageTests.swift» и «BranchCoverageTests.swift» закрывают дополнительные чистые ветви и вспомогательные сценарии.

4. Тесты auth-flow

4.1. Файл «AuthReducerFlowTests.swift» проверяет переходы между режимами login, register, verification pending и forgot password.

4.2. Проверяются ветви успешного входа, регистрации, подтверждения почты, повторной отправки письма подтверждения, отправки письма восстановления и завершения сброса пароля.

4.3. Проверяются обязательность email, пароля, verification token и reset token.

5. Тесты чатового сценария

5.1. Файл «ReducerFlowTests.swift» проверяет загрузку списка чатов, выбор активного чата, удаление чатов, начало сценария отправки сообщения и применение предложений событий.

5.2. Файл «ChatThreadFeatureFlowTests.swift» проверяет сброс состояния при смене токена или чата, загрузку истории, валидацию вложений, работу блока предложений и ветви ошибки отправки сообщения.

5.3. Файл «ChatAttachmentPickerFeatureTests.swift» проверяет одноразовый consent, открытие панели выбора вложений и добавление recent photo в compose-состояние.

6. Тесты календаря и ребалансировки

6.1. Файл «CalendarReducerFlowTests.swift» проверяет создание, редактирование и удаление

событий, перезагрузку событий по дате, сохранение ошибок и работу day-level «Resource Battery».

6.2. Файл «RebalanceFeatureFlowTests.swift» проверяет загрузку доступных дней, подгрузку Default Prompt, выбор дней, получение preview и применение ребалансировки.

7. Тесты навигации и визуального слоя

7.1. Файл «HomeProfileNavigationFeatureTests.swift» проверяет переходы между Home, Calendar, Achievements и Profile.

7.2. Файл «ViewSmokeTests.swift» проверяет создание и базовый рендер ключевых экранов и состояний: Home, Calendar, Profile, Achievements, Chat UI и Rebalance.

8. Результаты автоматизированных испытаний

8.1. Успешно выполнено 82 теста, сгруппированных в 21 suite.

8.2. Все тесты завершились со статусом «TEST SUCCEEDED».

8.3. В интерфейсе Xcode общее покрытие клиентского приложения отображается как 58%. Детализированный отчёт xccov для этого же результата испытаний составляет 58,28%, что соответствует 8482 покрытым строкам из 14555 исполняемых строк.

8.4. Критические участки чатового сценария покрыты существенно выше среднего: «ChatThreadSupport+Apply.swift» — 96,72%, «ChatThreadSupport+Send.swift» — 92,53%.

8.5. Вспомогательная логика списка чатов и типов ребалансировки также покрыта выше среднего: «ChatListFeature+Support.swift» — 94,44%, «RebalanceFeature+Types.swift» — 100,00%.

8.6. Значение общего покрытия ниже покрытия критических редьюсеров из-за того, что значительная часть строк приложения приходится на SwiftUI-представления и визуальные ветви, которые в текущем комплекте проверяются преимущественно smoke-тестами, а не полноценным UI automation.

6.3. Unit-тестирование серверной части

Unit-тесты проверяют корректность изолированных компонентов без взаимодействия с базой данных или внешними сервисами.

Инструменты: pytest 8.0+, pytest-asyncio. Запуск из директории `sentinel-backend/` командой `venv/bin/pytest tests/unit/ -v`.

Покрываемые компоненты:

1. **test_security.py** — JWT: генерация, верификация, истёкшие и подделанные токены; хеширование паролей (bcrypt).

2. **test_schemas.py** — валидация Pydantic-схем событий и напоминаний: обязательные поля, ограничения длины, end_at > start_at, допустимые значения source.

3. **test_llm_service.py** — оркестрация tool-use: системный промт (тайм-зона, дата, ai_instructions), сериализация истории чата, обработка вызовов инструментов (create/update/delete/search), паттерн propose-before-apply, fallback при ошибках.

4. **test_llm_vision.py** — обработка изображений: включение URL в контент при включённом LLM_VISION_ENABLED, исключение при выключенном.

5. **test_rebalance_service.py** — сервис ребалансировки: обнаружение пересечений событий, неизменность is_fixed-событий, формирование промта (Resource Battery, диапазон дат), retry при провале валидации, возврат 422 при двух провалах подряд.

6. **test_chat_repository.py** — пагинация с cursor, форматирование истории сообщений для LLM, структура multimodal-контента (image_message, event_actions с accepted/rejected/pending).

7. **test_storage_service.py** — S3: формирование ключа объекта, возврат публичного URL, определение MIME-типа по расширению.

8. **test_achievement_repository.py** — CRUD методы репозитория достижений: все уровни, граничные случаи (not found → None, empty list).

9. **test_achievement_service.py** — логика начисления достижений: обработка источников событий (user/ai), типов (event/reminder), защита от повторной выдачи.

10. **test_auth_token_repository.py** — CRUD токенов верификации и сброса пароля: создание, поиск по значению, истечение срока действия.

11. **test_counter_repository.py** — операции со счётчиками активности: инкремент, upsert, граничные случаи (нулевое значение, несуществующий счётчик).

12. **test_email_service.py** — отправка писем через FastMail: верификация аккаунта, сброс пароля.

13. **test_event_repository.py** — CRUD событий: поиск по диапазону дат, фильтрация по пользователю, граничные случаи (not found → None, пустой список).

14. **test_idempotency_repository.py** — проверка уникальности ключей, возврат существующего события при повторном запросе, изоляция ключей между пользователями.

15. **test_user_repository.py** — поиск по email, создание пользователя, обновление профиля (timezone, locale, ai_instructions), удаление.

6.4. Интеграционное тестирование серверной части

Интеграционные тесты проверяют работу API «от запроса до базы данных» через реальное подключение к PostgreSQL. Каждый тест выполняется в транзакции, откатываемой после завершения.

Подготовка: запустить `docker compose up -d` для поднятия базы данных, заполнить `.env`. Запуск из директории `sentinel-backend/` командой `venv/bin/pytest tests/integration/ -v`.

Покрываемые модули:

1. **test_health.py** — GET /api/v1/health: ответ {status: «healthy»}, код 200.

2. **test_auth.py** — регистрация (успех, дубль email, валидация полей, короткий пароль), вход (успех, неверный пароль, несуществующий email).

3. **test_profile.py** — GET/PATCH/DELETE /auth/me: обновление timezone, locale, ai_instructions; удаление аккаунта с правильным и неверным паролем; токен недействителен после удаления.

4. **test_events.py** — CRUD событий и напоминаний: создание, список, фильтрация по временному диапазону, получение по ID, частичное обновление, удаление, изоляция данных между пользователями.

5. **test_chats.py** — CRUD чатов, история сообщений с курсорной пагинацией, отправка сообщения с вызовом LLM (search_events и create_event через tool-use), применение предложенных действий.

6. **test_images.py** — загрузка изображения: допустимые форматы (JPG, PNG, HEIC), отказ при недопустимом формате (GIF), сохранение URL в content_structured сообщения.

7. **test_rebalance.py** — dry-run: обязательные поля, пустой список событий, несколько дней, некорректный resource_battery; apply: обновление временных меток событий, попытка сдвинуть is_fixed-событие (400).

8. **test_achievements.py** — накопление счётчиков (total_events, ai_events, total_reminders), разблокировка уровней при достижении порогов, защита от повторной выдачи, изоляция между пользователями.

9. **test_idempotency.py** — повторный запрос с тем же X-Idempotency-Key возвращает исходный объект; разные ключи создают разные события; изоляция ключей между пользователями.

10. **test_sync.py** — пакетная синхронизация: полный batch (создание, обновление, удаление), превышение лимита 500 (422), пустой body без операций.

6.5. Функциональные проверки серверной части

Функциональные проверки выполняются вручную через Swagger UI. Продуктивная среда: `https://sentinel-ai.tech/docs`. Локальная среда: `http://localhost:8000/docs` при запущенном сервисе.

6.5.1. Аутентификация и управление профилем

№	Действие	Входные данные	Ожидаемый результат
1	POST /auth/register	Корректные email, пароль >= 8 символов, тайм-зона	201, объект пользователя
2	POST /auth/register	Email уже зарегистрирован	409 CONFLICT
3	POST /auth/register	Отсутствует поле email	422 VALIDATION_ERROR
4	POST /auth/register	Пароль < 8 символов	422 VALIDATION_ERROR
5	POST /auth/login	Верные email и пароль	200, поле access_token
6	POST /auth/login	Неверный пароль	401 UNAUTHORIZED
7	POST /auth/login	Несуществующий email	401 UNAUTHORIZED
8	GET /auth/me	Bearer JWT	200, профиль пользователя
9	GET /auth/me	Без заголовка Authorization	401 UNAUTHORIZED
10	PATCH /auth/me	{timezone: «Europe/Moscow»}	200, обновлённый профиль
11	PATCH /auth/me	{ai_instructions: «…»}	200, поле ai_instructions обновлено
12	PATCH /auth/me	ai_instructions длиной > 500 символов	422 VALIDATION_ERROR
13	DELETE /auth/me	Верный пароль	204 No Content
14	DELETE /auth/me	Неверный пароль	400 BAD_REQUEST
15	GET /auth/me после DELETE	Ранее выданный JWT	401 UNAUTHORIZED

Таблица 1. Тест-кейсы: аутентификация и профиль

6.5.2. Управление событиями

№	Действие	Входные данные	Ожидаемый результат
1	GET /events	Bearer JWT, без параметров	200, {items, total}
2	GET /events?date_from&date_to	Диапазон дат ISO-8601	200, только события в диапазоне
3	POST /events	title, start_at, end_at	201, объект события с id
4	POST /events — напоминание	title, start_at (без end_at)	201, end_at = null

№	Действие	Входные данные	Ожидаемый результат
5	POST /events	Без поля title	422 VALIDATION_ERROR
6	POST /events	end_at раньше start_at	422 VALIDATION_ERROR
7	POST /events	Без заголовка Authorization	401 UNAUTHORIZED
8	POST /events + X-Idempotency-Key	Первый запрос	201, новое событие
9	POST /events + тот же ключ	Повтор запроса	200, тот же объект без дубля
10	GET /events/{id}	Существующий ID	200, объект события
11	GET /events/{id}	Несуществующий ID	404 NOT_FOUND
12	PATCH /events/{id}	{title: «Новое название»}	200, обновлённый объект
13	PATCH /events/{id}	ID события другого пользователя	403 FORBIDDEN
14	DELETE /events/{id}	Существующий ID	204 No Content
15	DELETE /events/{id}	ID события другого пользователя	403 FORBIDDEN
16	POST /events/sync	{upserts: […], deletes: […]}	200, {created, updated, deleted}
17	POST /events/sync	upserts: > 500 элементов	422 VALIDATION_ERROR

Таблица 2. Тест-кейсы: события и синхронизация

6.5.3. Диалоги, LLM и изображения

№	Действие	Входные данные	Ожидаемый результат
1	POST /chats	{title: «Мой чат»}	201, объект чата
2	GET /chats	Bearer JWT	200, список чатов
3	DELETE /chats/{id}	Существующий ID	204 No Content
4	GET /chats/{id}/messages	Параметры limit, before	200, {items, has_more}
5	POST /chats/{id}/messages	{role:«user», content_text:«…»}	200, ответ ассистента
6	POST /chats/{id}/messages — создание события	«Добавь встречу завтра в 10»	200, content_structured.type = event_actions
7	POST …/apply	{accepted_indices: [0]}	200, событие создано в базе данных
8	POST …/apply	{accepted_indices: []} — отклонить всё	200, события не созданы
9	POST …/apply чужого пользователя	Сообщение другого пользователя	403 FORBIDDEN

№	Действие	Входные данные	Ожидаемый результат
10	POST /chats/{id}/upload	JPG-файл <= 10 МБ	200, {url, filename, mime_type}
11	POST /chats/{id}/upload	GIF-файл	422 VALIDATION_ERROR
12	GET /chats/{id}/messages после upload	—	200, content_structured.type = image_message

Таблица 3. Тест-кейсы: чаты и LLM

6.5.4. Ребалансировка расписания

№	Действие	Входные данные	Ожидаемый результат
1	POST /events/rebalance	date_from, date_to, resource_battery	200, {proposed, summary, changed_count, unchanged_count}
2	POST /events/rebalance	Нет событий в диапазоне	200, proposed = []
3	POST /events/rebalance	resource_battery значение 1.5 (> 1.0)	422 VALIDATION_ERROR
4	POST /events/rebalance	days: [] (пустой список)	422 VALIDATION_ERROR
5	POST /events/rebalance/apply	{changes: [{id, start_at, end_at}]}	204 No Content, времена обновлены
6	POST /events/rebalance/apply	is_fixed-событие в списке changes	400 BAD_REQUEST
7	POST /events/rebalance	Без заголовка Authorization	401 UNAUTHORIZED

Таблица 4. Тест-кейсы: ребалансировка

6.5.5. Достижения

№	Действие	Входные данные	Ожидаемый результат
1	GET /achievements	Новый пользователь	200, все уровни locked, current_value = 0
2	GET /achievements после создания события	—	200, total_events увеличен на 1
3	GET /achievements после N событий	N равно target_value уровня 1	200, уровень 1 unlocked = true
4	GET /achievements повторно	Достижение уже выдано	200, earned_at не изменяется
5	GET /achievements пользователя B	Пользователи A и B изолированы	200, данные только пользователя B

Таблица 5. Тест-кейсы: достижения

6.5.6. Надёжность и граничные случаи

№	Действие	Входные данные	Ожидаемый результат
1	Любой защищённый эндпоинт	Без заголовка Authorization	401 UNAUTHORIZED
2	Любой защищённый эндпоинт	Истёкший JWT	401 UNAUTHORIZED
3	GET /events/{id} чужого пользователя	ID события пользователя A	403 FORBIDDEN
4	Любой эндпоинт с невалидным JSON	Нарушение схемы Pydantic	422, details с описанием ошибки
5	GET /health	Без авторизации	200, {status: «healthy»}
6	POST /events после DELETE /auth/me	JWT удалённого пользователя	401 UNAUTHORIZED

Таблица 6. Тест-кейсы: надёжность

6.6. Тестирование функциональности iOS-клиента

Функциональные испытания выполнялись путём сопоставления автоматизированных flow-тестов, smoke-проверок визуального слоя и фактической структуры представлений и редьюсеров проекта с основными пользовательскими сценариями Sentinel.

1. Сценарий авторизации

1.1. Проверялись вход по email и паролю, регистрация в два шага, подтверждение почты, повторная отправка письма подтверждения и восстановление пароля.

1.2. Проверялись обязательность полей и корректный сброс временных полей при переключении между режимами auth-flow.

1.3. Результат: сценарий авторизации подтверждён тестами «AuthReducerFlowTests.swift» и производными состояниями «AchievementAndAuthStateTests.swift».

2. Сценарий работы со списком чатов и активным диалогом

2.1. Проверялись загрузка списка чатов, выбор первого активного чата, удаление чата, переход к новому чату и обновление активного идентификатора.

2.2. Проверялась загрузка истории сообщений, включая подгрузку более ранних сообщений и ветви ошибки.

2.3. Результат: сценарий подтверждён тестами «ReducerFlowTests.swift» и «ChatThreadFeatureFlowTests.swift».

3. Сценарий отправки сообщения и вложений

3.1. Проверялись отправка текстового сообщения, автоматическое создание чата при первой

отправке, подготовка и отправка вложения.

3.2. Проверялись однократное согласие на серверную обработку вложений, загрузка recent photos, добавление изображений в compose-состояние, а также ограничения на размер и количество вложений.

3.3. Проверялись ветви неуспешной отправки сообщения, при которых сообщение остаётся в ленте и переходит в состояние failed.

3.4. Результат: сценарий подтверждён тестами «ChatThreadFeatureFlowTests.swift», «ChatAttachmentPickerFeatureTests.swift», «ReducerFlowTests.swift» и доменными моделями «ChatDomainModelTests.swift».

4. Сценарий применения предложенных событий

4.1. Проверялись формирование блока предложений событий, выбор предложений, вычисление конфликтов и построение плана применения.

4.2. Проверялись отправка на сервер информации о выбранных действиях, обновление сообщения ассистента после применения и последующая синхронизация календарных данных.

4.3. Результат: критический сценарий «чат → предложения событий → подтверждение → календарь» подтверждён тестами «ReducerFlowTests.swift», «ChatThreadFeatureFlowTests.swift» и «ChatThreadSupportTests.swift».

5. Сценарий календаря и ручного редактирования событий

5.1. Проверялись загрузка событий по диапазону дат, переключение между днями и месяцами, открытие редактора события, сохранение и удаление записи.

5.2. Проверялись вычисление строк повестки дня, признаки конфликтов, отображение reminder/event и построение day-level battery request.

5.3. Результат: сценарий подтверждён тестами «CalendarReducerFlowTests.swift» и «CalendarStateDerivedTests.swift».

6. Сценарий Profile, legal-flow и настроек пользователя

6.1. Проверялись загрузка пользовательских настроек, редактирование Default Prompt, автосохранение Default Prompt и изменение выбранного окружения.

6.2. Проверялись наличие legal-flow, состава юридических документов и корректность URL для открываемых документов.

6.3. Проверялись выход из учётной записи и удаление учётной записи с обязательным подтверждением паролем.

6.4. Результат: сценарий подтверждён тестами «AppConfigurationAndSettingsTests.swift»,

«DefaultPromptEnvelopeTests.swift», «HomeProfileNavigationFeatureTests.swift» и анализом редьюсера ProfileFeature.

7. Сценарий достижений и домашнего экрана

7.1. Проверялись построение карточек домашнего экрана, вычисляемые свойства расписания и «Resource Battery», а также агрегированные свойства достижений.

7.2. Проверялись переходы с Home к календарю и достижениям, а из Profile — к достижениям.

7.3. Результат: сценарий подтверждён тестами «HomeStateDerivedTests.swift», «AchievementAndAuthStateTests.swift» и «HomeProfileNavigationFeatureTests.swift».

8. Сценарий Rebalance

8.1. Проверялись загрузка доступных дней, автоматическая подгрузка Default Prompt, выбор дней, preview, refresh и apply.

8.2. Проверялась синхронизация календарных данных после применения ребалансировки и обновление соответствующих состояний.

8.3. Результат: сценарий подтверждён тестами «RebalanceFeatureFlowTests.swift».

9. Ограничения функциональных испытаний

9.1. Отдельный набор XCUITest-сценариев на момент проведения испытаний отсутствует.

9.2. Полноценные end-to-end проверки камеры, системных разрешений и реального календаря устройства не включены в текущий автоматизированный набор и должны наращиваться в следующих итерациях.

6.7. Сквозное функциональное тестирование комплекса

Сквозные функциональные проверки выполнялись путём сопоставления клиентских flow-тестов, серверных интеграционных тестов, API-контрактов и логики фактического взаимодействия между компонентами.

1. Сценарий аутентификации и профиля

1.1. Проверялись регистрация, вход, получение профиля, обновление тайм-зоны и ai_instructions, выход и удаление аккаунта.

1.2. Ожидаемый результат: клиент корректно хранит JWT, сервер возвращает профиль текущего пользователя, после удаления аккаунта прежний JWT становится недействительным.

2. Сценарий отправки сообщения и вложений

2.1. Проверялись отправка текстового сообщения, автоматическое создание чата при необходимости, загрузка изображения, использование consent и появление multimodal-контента в истории чата.

2.2. Ожидаемый результат: клиент отображает сообщение в ленте, сервер сохраняет сообщение и возвращает ответ ассистента; изображение загружается в объектное хранилище и участвует в обработке запроса.

3. Сценарий предложений событий

3.1. Проверялись получение блока `event_actions`, выбор одного или нескольких предложений, отклонение части предложений и apply-операция.

3.2. Ожидаемый результат: до apply действия находятся в состоянии pending; после apply сервер сохраняет принятые изменения, а клиент обновляет сообщение ассистента и локальные календарные данные.

4. Сценарий синхронизации календаря

4.1. Проверялись создание и редактирование события на сервере, повторная загрузка его клиентом и локальный экспорт в EventKit без дублирования.

4.2. Ожидаемый результат: серверное событие отображается на клиенте, а повторные синхронизации не создают дублей в локальном календаре.

5. Сценарий ребалансировки

5.1. Проверялись загрузка доступных дней, передача Default Prompt и day-level данных, получение preview и apply изменений.

5.2. Ожидаемый результат: dry-run не изменяет серверные данные, apply изменяет только подтверждённые события, fixed-события не нарушаются.

6. Сценарий достижений

6.1. Проверялись накопление счётчиков на сервере, получение групп достижений клиентом и корректное отображение прогресса.

6.2. Ожидаемый результат: клиент отображает только данные текущего пользователя, повторная загрузка не создаёт ложных повторных выдач достижений.

7. Общий результат

7.1. Сквозной сценарий «чат → предложения событий → подтверждение → календарь» подтверждён совокупностью серверных и клиентских автоматизированных проверок и соответствует основной продуктовой логике Sentinel.

7.2. Сквозной сценарий «профиль → Default Prompt → чат / ребалансировка» подтверждён: значение Default Prompt сохраняется на клиенте и учитывается при формировании сетевых запросов.

6.8. Проверка требований к надёжности

Проверка требований к надёжности iOS-клиента.

Проверка надежности выполнялась на основе автоматизированных тестов редьюсеров, доменных моделей и настроек приложения, анализирующих ветви валидации, error-handling и восстановления состояния.

1. Проверка устойчивости к некорректному вводу

1.1. Проверялась обязательность email и пароля в auth-сценариях.

1.2. Проверялась обязательность verification token и reset token.

1.3. Проверялась обязательность заголовка события при сохранении.

1.4. Проверялась реакция на слишком большой файл и превышение лимита вложений.

1.5. Результат: все перечисленные ветви обрабатываются контролируемо и завершаются сообщением пользователю без аварийного завершения модуля.

2. Проверка поведения при ошибках операций

2.1. Проверялась реакция на неуспешную отправку сообщения.

2.2. Проверялась реакция на ошибку удаления события.

2.3. Проверялась реакция на ошибку сохранения события.

2.4. Проверялась ветвь login с ответом verification required.

2.5. Результат: ошибки переводят модули в предсказуемое состояние, сохраняют необходимые пользовательские данные и формируют читаемое сообщение об ошибке.

3. Проверка восстановления и сброса состояния

3.1. Проверялась попытка восстановления сессии при отсутствии сохранённых данных.

3.2. Проверялся полный сброс чувствительного состояния при потере access token.

3.3. Проверялось сохранение и повторная загрузка AppSettings, включая Default Prompt, selectedEnvironment и сведения о недавно активном чате.

3.4. Результат: логика восстановления и сброса состояния подтверждена тестами «AuthReducerFlowTests.swift», «AppConfigurationAndSettingsTests.swift» и «ChatThreadFeatureFlowTests.swift».

4. Проверка целостности локальных и серверных данных

4.1. Проверялись сериализация и десериализация пользовательской сессии.

4.2. Проверялись транспортные контракты событий, чатов, сообщений и достижений.

4.3. Проверялись построение плана применения выбранных предложений событий и расчёт диапазона повторной синхронизации.

4.4. Результат: ключевые механизмы целостности подтверждены тестами «SessionModelTests.swift», «DTOCodingTests.swift», «APIModelConverterTests.swift» и

«ChatDomainModelTests.swift».

5. Общий вывод по надежности

5.1. Проверенные сценарии подтверждают устойчивость критических частей доменной логики к типовым ошибкам ввода и отказам внешних операций.

5.2. Ограничением текущего этапа остаётся отсутствие отдельного набора длительных нагрузочных испытаний и полного UI automation.

5.3. Указанное ограничение не отменяет результата выполненного модульного и flow-тестирования, но должно учитываться при дальнейшем развитии тестовой базы проекта.

Проверка требований к надёжности серверной части.

Проверяется в рамках интеграционного тестирования: все тесты проходят при штатной работе PostgreSQL и сервиса. Валидация входных данных через Pydantic v2 подтверждается тест-кейсами раздела «Надёжность и граничные случаи».

6.8.1. Время восстановления после отказа

Проверяется путём намеренной остановки контейнера приложения командой `docker compose stop app` и последующего перезапуска `docker compose start app`. Результат контролируется через GET /api/v1/health: ответ `{status: "healthy"}` должен появиться в течение 5 минут.

6.8.2. Отказы из-за некорректных действий

Проверяется тест-кейсами из таблицы «Надёжность и граничные случаи»: 401 при отсутствии или истечении JWT, 403 при доступе к чужим ресурсам, 422 при нарушении схемы входных данных.

Проверка требований к надёжности взаимодействия компонентов.

1. Проверялась согласованность передачи ошибок 401, 403, 404, 422 и 429 от серверной части к клиентскому UI.
2. Проверялась устойчивость сквозного сценария к повторной отправке запросов, сбоям синхронизации и повторному получению серверных данных после перезапуска клиента или сервиса.
3. Проверялась сохранность связки «серверное событие — локальное событие EventKit» и отсутствие дублей при повторной синхронизации.
4. Общий вывод: проверенные сценарии подтверждают устойчивость интеграционного контура к типовым ошибкам ввода, сетевым отказам и повторным операциям.

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. Apple Developer Documentation. EventKit. // Электронный ресурс. URL: https://developer.apple.com/documentation/eventkit (дата обращения 29.04.26).
15. Apple Developer Documentation. SwiftUI. // Электронный ресурс. URL: https://developer.apple.com/documentation/swiftui (дата обращения 29.04.26).

16. Apple Developer Documentation. UserDefaults. // Электронный ресурс. URL: https://developer.apple.com/documentation/foundation/userdefaults (дата обращения 29.04.26).
17. Apple Developer Documentation. Keychain Services. // Электронный ресурс. URL: https://developer.apple.com/documentation/security/keychain_services (дата обращения 29.04.26).
18. Apple Developer Documentation. SFSafariViewController. // Электронный ресурс. URL: https://developer.apple.com/documentation/safariservices/sfsafariviewcontroller (дата обращения 29.04.26).
19. Swift.org. Swift Testing. // Электронный ресурс. URL: https://www.swift.org/documentation/testing/ (дата обращения 29.04.26).
20. Point-Free. The Composable Architecture. // Электронный ресурс. URL: https://github.com/pointfreeco/swift-composable-architecture (дата обращения 29.04.26).
21. Исходный код проекта Sentinel iOS // GitHub репозиторий. – URL: https://github.com/cutesp1rit/Sentinel_coursework/tree/main/sentinel-ios (дата обращения 29.04.26).

ПРИЛОЖЕНИЕ 2. ТЕРМИНЫ И СОКРАЩЕНИЯ

1. API (Application Programming Interface) — программный интерфейс взаимодействия клиента с серверной частью по HTTPS и JSON.
2. AppSettings — локальная структура пользовательских настроек приложения, включающая Default Prompt, выбранное окружение и сведения о недавно активном чате.
3. Default Prompt — пользовательский текстовый шаблон, автоматически добавляемый к каждому сообщению в чате и к каждому запросу ребалансировки.
4. EventKit — системный фреймворк Apple для доступа к календарям и событиям устройства.
5. Flow-тест — автоматизированный тест, проверяющий последовательность действий, эффектов и переходов состояния внутри пользовательского сценария.
6. JSON (JavaScript Object Notation) — формат представления структурированных данных, используемый в сетевых контрактах приложения.
7. Keychain — защищённое системное хранилище Apple для токенов и иных чувствительных данных.
8. Rebalance — функция приложения Sentinel для получения и применения предложения по перераспределению событий на выбранном диапазоне дней.
9. Resource Battery — вычисляемая модель загруженности расписания пользователя на ближайший период или на отдельный день.
10. SFSafariViewController — системный компонент iOS для отображения веб-страниц внутри приложения.
11. Smoke-тест — быстрая проверка того, что экран или базовый сценарий может быть создан и отрисован без ошибки.
12. SwiftData — технология локального хранения данных, используемая в проекте для записей синхронизации календаря.
13. Swift Testing — фреймворк тестирования на Swift, применяемый в таргете SentinelTests.
14. SwiftUI — декларативный фреймворк Apple для построения пользовательского интерфейса.
15. TCA (The Composable Architecture) — архитектурный подход с однонаправленным потоком данных, лежащий в основе клиентского приложения Sentinel.
16. TestStore — инструмент TCA для тестирования редьюсеров, их эффектов и переходов состояний.
17. UserDefaults — системное хранилище настроек пользователя, используемое в проекте для AppSettings и service-флагов.
18. View — экран или визуальный компонент пользовательского интерфейса в SwiftUI.

19. Reducer — модуль бизнес-логики ТСА, преобразующий состояние приложения в ответ на действия пользователя и эффекты.

ПРИЛОЖЕНИЕ А. РЕЗУЛЬТАТЫ АВТОМАТИЧЕСКОГО ТЕСТИРОВАНИЯ

Ниже приведены результаты запуска автоматических тестов командой:

```
venv/bin/pytest tests/ --cov=app --cov-report=term-missing
```

Итого: **385 тестов**, из которых unit-тесты — 15 модулей, интеграционные тесты — 10 модулей. Все тесты пройдены успешно. Суммарное покрытие кода — **87%**.

```
================================================================================ tests coverage ================================================================================
_____________________________________________________________ coverage: platform darwin, python 3.10.15-final-0 _____________________________________________________________

Name                                                                      Stmts   Miss  Cover   Missing
-------------------------------------------------------------------------------------------------------
app/__init__.py                                                               0      0   100%
app/api/__init__.py                                                           0      0   100%
app/api/dependencies.py                                                      19      3    84%   32-38
app/api/v1/__init__.py                                                        2      0   100%
app/api/v1/achievements.py                                                   25      9    64%   24-44, 60
app/api/v1/auth.py                                                          113     60    47%   40-58, 68-80, 91-108, 120-134, 146-155, 166-183
app/api/v1/chats.py                                                         114     63    45%   35, 67-72, 82-85, 98-117, 141-162, 169-171, 213-262
app/api/v1/events.py                                                         79     31    61%   39, 46, 59-67, 72-84, 103-113, 133-135, 147-150, 161-168
app/api/v1/rebalance.py                                                      38      7    82%   39-41, 51, 69-74
app/core/__init__.py                                                          0      0   100%
app/core/config.py                                                           51      2    96%   28, 31
app/core/limiter.py                                                          17      0   100%
app/core/schemas/__init__.py                                                  3      0   100%
app/core/schemas/achievement.py                                              22      0   100%
app/core/schemas/chat.py                                                     74      0   100%
app/core/schemas/event.py                                                    63      0   100%
app/core/schemas/rebalance.py                                                32      0   100%
app/core/schemas/user.py                                                     41      0   100%
app/core/security.py                                                         28      0   100%
app/core/services/__init__.py                                                 0      0   100%
app/core/services/achievement_service.py                                     47      0   100%
app/core/services/email_service.py                                           14      0   100%
app/core/services/llm_service.py                                            138     23    83%   125-127, 170-171, 175, 182-188, 196, 206-208, 265, 269-271, 287, 291, 293-294, 300, 302-303
app/core/services/rebalance_service.py                                      241     27    89%   76, 131, 148, 173-174, 281-282, 294, 296, 348, 361-372, 393, 405-406, 431, 440, 443, 449, 451
app/core/services/storage_service.py                                         12      0   100%
app/infrastructure/__init__.py                                                0      0   100%
app/infrastructure/database/__init__.py                                       0      0   100%
app/infrastructure/database/base.py                                          12      4    67%   31-35
app/infrastructure/database/models/__init__.py                                7      0   100%
app/infrastructure/database/models/achievement.py                            36      0   100%
app/infrastructure/database/models/auth_token.py                             15      0   100%
app/infrastructure/database/models/chat.py                                   26      0   100%
app/infrastructure/database/models/event.py                                  22      0   100%
app/infrastructure/database/models/idempotency.py                            14      0   100%
app/infrastructure/database/models/user.py                                   23      0   100%
app/infrastructure/database/repositories/__init__.py                          6      0   100%
app/infrastructure/database/repositories/achievement_repository.py           28      0   100%
app/infrastructure/database/repositories/auth_token_repository.py            20      0   100%
app/infrastructure/database/repositories/chat_repository.py                 114      0   100%
app/infrastructure/database/repositories/counter_repository.py               24      0   100%
app/infrastructure/database/repositories/event_repository.py                 88      0   100%
app/infrastructure/database/repositories/idempotency_repository.py           22      0   100%
app/infrastructure/database/repositories/user_repository.py                  45      0   100%
app/infrastructure/external/__init__.py                                       0      0   100%
app/infrastructure/external/openai_client.py                                  4      0   100%
app/main.py                                                                  43      1    98%   79
app/scripts/__init__.py                                                       0      0   100%
-------------------------------------------------------------------------------------------------------
TOTAL                                                                      1722    230    87%
========================================================================== 382 passed, 1 skipped, 17 warnings in 54.63s ==========================================================================
❯ tim-maks@172 sentinel-backend %
```

Рис. 1. Отчёт о покрытии кода тестами

Пониженное покрытие API роутов является известным техническим ограничением инструмента: Starlette TestClient запускает ASGI-приложение в отдельном anyio-треде, тогда как coverage.py отслеживает выполнение через sys.settrace() только в текущем треде. После первого await внутри route-функции coverage теряет трек, поэтому строки после него не отображаются как покрытые, хотя код реально выполняется — все тесты проходят.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					